Lab 10 : Digital CPU Foundations (CPU Datapath and Control)

Objectives

In this lab, you will integrate the individual components created in Lab 9 to build a complete, single-cycle CPU datapath. You will then design a control unit to interpret instructions and orchestrate the datapath's operations. Finally, you will test your CPU with a simple program.

Lab 10.1 : Assembling the Datapath

The datapath consists of all the components that process and transfer data, connected by busses and multiplexers.

1. Prepare the Data Memory (EEPROM):

1.1. Open your dataRAM.dig file (created in Lab 9).

1.2. Replace the RAM component with an EEPROM component (Components → Memory → EEPROM), keeping the same Address Bits=6, Data Bits=4, and the same pin connections (A, Din, D, str, ld, C) as before. EEPROM behaves identically to RAM during simulation, but — unlike RAM — its content can be edited directly in edit mode, without needing to start the simulation first.

1.3. Right-click the EEPROM component → "Edit Content", and put the value 5 at address 2. This value is saved with the file, so it will already be there every time you open dataRAM.

2. Create and Arrange Components:

2.1. Create a new file, cpu.dig.

2.2. Add your subcircuits: programCounter, programROM, registerFile, miniALU, dataRAM.

3. Connect Instruction Fetch & Decode Logic:

3.1. Connect the programCounter output to the programROM address input.

3.2. Instruction format is now **14 bits** wide, matching the programROM Data Bits = 14 from Lab 9:

Opcode(3) + Type_Bit(1) + Reg1_Addr(3) + Reg2_Addr(3) + Address_Field(4) = 14 bits

3.3. Primary Splitter: Add a Splitter/Merger. Set Input Splitting to 14 and Output Splitting to 4,3,3,1,3 (in this order: Address_Field bits 3-0, Reg2_Addr bits 6-4, Reg1_Addr bits 9-7, Type_Bit bit 10, Opcode bits 13-11). Note: in Digital, the first group listed in Output Splitting becomes the lowest-order bits, so Address_Field must be listed first to land at bits 3-0.

3.4. Label the splitter outputs directly (no extra merger needed). The splitter's output pins will appear top-to-bottom in this order: Address_Field (4-bit), Reg2_Addr (3-bit), Reg1_Addr (3-bit), Type_Bit (1-bit), Opcode (3-bit).

3.5. **Address Field Extension (new step):** dataRAM and programCounter use 6-bit addresses (Lab 9), but Address_Field is only 4 bits. Add a Constant Value component (Components -> Wires -> Constant Value)— (Value=0, Data Bits=2) and a Splitter/Merger configured as a merger with Input Splitting = 4,2 (Address_Field on the top input, the 2-bit Constant on the bottom input this places Address_Field at bits 3-0 and the zero constant at bits 5-4, preserving Address_Field's value). Label the result Address_Field_Ext (6-bit).

4. Connect Register File and ALU:

4.1. Connect Reg1_Addr (3-bit) to the Rw and Ra inputs of the registerFile.

4.2. Connect Reg2_Addr (3-bit) to the Rb input.

4.3. Connect registerFile.Da to miniALU.A and dataRAM.Din.

4.4. Connect registerFile.Db to miniALU.B.

5. Connect Data Memory Path:

5.1. Connect Address_Field_Ext (6-bit) to the A (address) input of dataRAM.

6. Implement Write-Back Logic (3-Source MUX):

- Add a 4-bit Multiplexer with 2 Number of Selector Bits.
- Connect miniALU.Result → in_0.
- Connect dataRAM.D → in_1.
- Connect registerFile.Db → in_2.
- Tie in_3 to a 4-bit Constant Value (0)
- Connect the MUX out → registerFile.Din.
- The 2-bit sel input is the control signal RegWriteSource.

7. **Update PC / Branching Logic:**

• Fan out the same programCounter.OUT wire used in step 3.1 (the one feeding programROM's address input) so it also feeds a second destination: a 6-bit Adder.

• Connect programCounter.OUT to input A, and a Constant Value component (Value=1, Data Bits=6) to input B. Connect the Adder's C (carry-in) pin to GND (Components → Wires → Ground).

• Add a 6-bit Multiplexer to choose the next PC value. Connect the Adder's output (PC+1) to in_0, and Address_Field_Ext to in_1.

• Connect the MUX out to the programCounter in pin.

• The sel input of this MUX is the control signal PCSource: sel=0 selects PC+1 (in_0), sel=1 selects Address_Field_Ext (in_1) — matching the truth table (PCSource=0 normal, PCSource=1 branch taken).

Lab 10.2 : Designing the Full Control Unit

The control unit is the brain. It takes the instruction's Opcode and Type_Bit to generate all the necessary control signals.

1. Identify Control Signals:

- RegWrite, MemRead, MemWrite, FlagWrite, PCSource (all 1 bit)
- RegWriteSource (2 bits)

2. Full Control Logic Truth Table (Opcode is now 3 bits; BR added):

Opcode	Type	Instruction	RegWrite	MemRead	MemWrite	RegWriteSource	FlagWrite	PCSource
000	0	LOAD (Reg)	1	0	0	10 (Reg Db)	0	0
000	1	LOAD (Mem)	1	1	0	01 (Memory)	0	0
001	X	STORE	0	0	1	XX	0	0
010	X	ADD	1	0	0	00 (ALU)	1	0
011	X	BZ (branch if zero)	0	0	0	XX	0	Zero_Flag
100	X	BR (branch always) — NEW	0	0	0	XX	0	1 (always)

3. Implement the Control Unit:

3.1. Create/Open your controlUnit.dig circuit.

3.2. Add three inputs: Opcode (3 bits), Type_Bit (1 bit), Zero_Flag (1 bit).

3.3. Use a Decoder on the Opcode input (Components → Plexers → Decoder. Right-click it and set Selector Bits = 3) and label the outputs: y0 = LOAD, y1 = STORE, y2 = ADD, y3 = BZ, y4 = BR. (y5–y7 are unused opcodes; leave them unconnected.)

3.4. Use Logic Gates to Generate Signals based on the table:

• RegWrite = LOAD OR ADD

• MemRead = LOAD AND Type_Bit

• MemWrite = STORE (a direct wire; no gate needed)

• FlagWrite = ADD (a direct wire; no gate needed)

• PCSource = (BZ AND Zero_Flag) OR BR

• RegWriteSource Logic (2-bit bus):

sel0 = LOAD AND Type_Bit

sel1 = LOAD AND (NOT Type_Bit)

Combine sel0 and sel1 into a 2-bit bus using a Splitter/Merger (Input Splitting = 1,1; Output Splitting = 2). Connect sel0 to the top input and sel1 to the bottom input.

3.5. Add six Output components (Components → IO → Output), one for each control signal — RegWrite, MemRead, MemWrite, FlagWrite, PCSource (1-bit each), and RegWriteSource (2-bit) — and connect each directly to the gate/wire that drives it from step 3.4.

3.6. Test the Control Unit:

3.6.1. Add a Test component (Components → Misc. → Test) to your controlUnit.dig canvas. Right-click it and select "Edit Detached" to open the test vector editor.

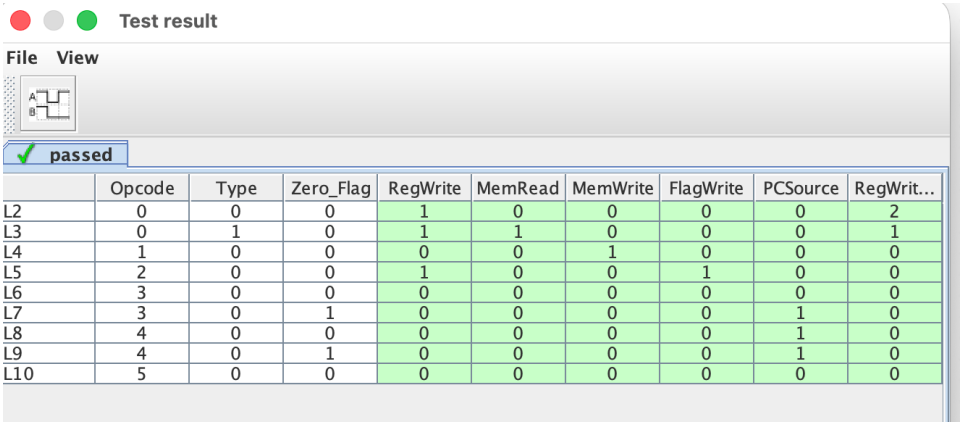
3.6.2. Type the header row and one row per test case, separating columns with spaces only. Prefix every binary value with 0b:

Opcode	Type	Zero_Flag	RegWrite	MemRead	MemWrite	FlagWrite	PCSource	RegWriteSource
0b000	0	0	1	0	0	0	0	0b10
0b000	1	0	1	1	0	0	0	0b01
0b001	0	0	0	0	1	0	0	0b00
0b010	0	0	1	0	0	1	0	0b00
0b011	0	0	0	0	0	0	0	0b00
0b011	0	1	0	0	0	0	1	0b00
0b100	0	0	0	0	0	0	1	0b00
0b100	0	1	0	0	0	0	1	0b00
0b101	0	0	0	0	0	0	0	0b00

3.6.3. Click "Run Tests" inside the editor, then click OK to save the test data back onto the canvas. Right-click the Test component on the canvas and choose "Run Tests" again to see the full result table.

3.6.4. A "passed" banner (green check) with every row highlighted green means your control logic is correct. A "failed" banner highlights the specific cell(s) that don't match, showing E: (expected) / F: (found) — use this to trace back to the gate producing that signal.

3.6.5. The last three rows (Opcode = BR twice with different Zero_Flag, and an unused opcode 101) specifically catch two common mistakes: PCSource incorrectly depending on Zero_Flag during BR, and any signal firing on an opcode that shouldn't exist.



	Opcode	Type	Zero_Flag	RegWrite	MemRead	MemWrite	FlagWrite	PCSource	RegWrit...
L2	0	0	0	1	0	0	0	0	2
L3	0	1	0	1	1	0	0	0	1
L4	1	0	0	0	0	1	0	0	0
L5	2	0	0	1	0	0	1	0	0
L6	3	0	0	0	0	0	0	0	0
L7	3	0	1	0	0	0	0	1	0
L8	4	0	0	0	0	0	0	1	0
L9	4	0	1	0	0	0	0	1	0
L10	5	0	0	0	0	0	0	0	0

Lab 10.3 : Final Assembly and Testing

1. Connecting Inputs to the Control Unit:

1.1. Place the controlUnit Subcircuit: Drag your controlUnit.dig file onto your cpu.dig canvas.

1.2. Connect the Opcode Input:

1.2.1. Purpose: These three bits tell the Control Unit what kind of operation to perform (LOAD, STORE, ADD, BZ, or BR).

1.2.2. Connection: Find the 3-bit Opcode output (bits 13-11) from your primary instruction splitter. Connect this 3-bit wire directly to the Opcode input pin on your controlUnit.

1.3. Connect the Type_Bit Input:

1.3.1. Purpose: This single bit distinguishes between the two different LOAD instructions. The Control Unit needs to know if it should get data from memory (Type=1) or another register (Type=0).

1.3.2. Connection: Find the 1-bit Type_Bit output (bit 10) from your primary instruction splitter. Connect this wire to the Type_Bit input pin on your controlUnit.

1.4. Connect the Zero_Flag Input:

1.4.1. Purpose: This input tells the control logic whether the result of the last ADD operation was zero, which is essential for the BZ (Branch if Zero) instruction.

1.4.2. Connection: Find the 1-bit Zero_Flag output from your miniALU subcircuit. Connect this wire to the Zero_Flag input pin on your controlUnit.

2. Connecting Outputs from the Control Unit to the Datapath:

2.1. RegWrite → registerFile.we

2.1.1. Purpose: The write-enable signal for the Register File. When 1, the register file writes Din into the register addressed by Rw on the next clock tick.

2.1.2. Connection: Connect the RegWrite output from your controlUnit to the we input pin on your registerFile subcircuit.

2.2. MemRead → dataRAM.ld

2.2.1. Purpose: Tells the Data RAM to perform a read. When 1, a LOAD-from-memory instruction is executing, and the RAM outputs the data at the given address onto D.

2.2.2. Connection: Connect the MemRead output from the controlUnit to the ld input pin (MemRead) on your dataRAM subcircuit.

2.3. MemWrite → dataRAM.str

2.3.1. Purpose: Tells the Data RAM to perform a write. When 1, a STORE instruction is executing, and the RAM stores Din at the given address.

2.3.2. Connection: Connect the MemWrite output from the controlUnit to the str input pin on your dataRAM subcircuit.

2.4. FlagWrite → miniALU.FlagWrite

2.4.1. Purpose: Enables the register inside miniALU to latch the new Zero Flag status. Only active during an ADD instruction.

2.4.2. Connection: Connect the FlagWrite output from the controlUnit to the FlagWrite input pin on your miniALU subcircuit.

2.5. PCSource → Next PC MUX sel

2.5.1. Purpose: Chooses the next PC value. If 0, selects PC+1 (normal operation). If 1, selects Address_Field_Ext — a branch is taken (BZ with Zero_Flag = 1, or BR unconditionally).

2.5.2. Connection: Connect the PCSource output from the controlUnit to the sel input pin of the multiplexer that feeds into your programCounter.

2.6. RegWriteSource → Write-Back MUX sel

2.6.1. Purpose: This 2-bit signal selects which data gets written back into a register. 00 selects the ALU result, 01 selects data from memory, 10 selects data from another register.

2.6.2. Connection: Connect the 2-bit RegWriteSource output from the controlUnit to the 2-bit sel input of the 3-source multiplexer whose output goes to the registerFile's Din pin.

3. Prepare a Test Program:

3.1. Load Program into ROM: Right-click programROM → "Edit Content". Enter the following 14-bit instruction words:

Addr	Instruction	Encoding (14-bit hex)	Note
0	LOAD R1, Mem[2]	0x0482	R1 ← 5
1	LOAD R2, R1	0x0110	R2 ← 5
2	ADD R1, R2	0x10A0	R1 ← 10 (0xA), ZF = 0
3	STORE R1, Mem[3]	0x0883	Mem[3] ← 10
4	BR 0	0x2000	Jump back to address 0 (loop) — demonstrates unconditional branch

4. Add the Master Clock Signal:

4.1. In your main cpu.dig file, add a single Clock component (Components → IO → Clock Input). This will be the one and only clock source for the entire CPU.

4.2. Connect this master clock to every sequential component:

- Clock output → programCounter "CLK" input pin.
- Clock output → registerFile "Clock" input pin.
- Clock output → miniALU "Clock" input pin.
- Clock output → dataRAM "Clock" input pin.

4.3. Enable the Program Counter: To ensure the PC updates on every cycle, connect the master Clock output to the ld (load) pin of your programCounter as well.

4.4. Tie off the Clear Input: This design does not use an external reset, so connect the CLR pin of your programCounter to GND (Components → Wires → Ground) to keep it from floating.

6. Add Probes for Verification:

6.1. Add a Probe (Components → IO → Probe) on each of the following lines — this lets you observe values directly on the canvas without needing to right-click anything once the simulation is running:

• programCounter In and Out

- registerFile Da and Db
- miniALU Result

6. Run and Verify:

6.1. Start the simulation (press Play).

6.2. Use single clock steps to verify each instruction executes correctly and monitor each probe.

6.3. After instruction 4 (BR 0), confirm the PC returns to address 0, demonstrating the unconditional branch, and the program loops.

Lab 10.4 : Exercise — Write Your Own Assembly Program

Using the CPU (cpu.dig) you built in this lab, write and run an assembly program for the following pseudocode:

```
j = 0;
for (i = 10; i > 0; i--)
   j += 2;
```

1. Register and Memory Planning:

1.1. Decide which registers will hold i and j.

1.2. This CPU's instruction set has **no immediate-load and no subtract instruction**. You must therefore pre-load any constants you need (initial values, step values, decrement values) into Data RAM addresses, then use LOAD (Mem) to bring them into registers before the loop begins.

1.3. Implement i-- using ADD with a suitable two's-complement constant.

2. Write the Program:

2.1. Write your instruction sequence in assembly mnemonic form (e.g., LOAD R1, Mem[0]).

2.2. Encode each instruction into its 14-bit hex value using the instruction field format from Lab 10.1: Opcode(3) + Type_Bit(1) + Reg1_Addr(3) + Reg2_Addr(3) + Address_Field(4).

3. Run and Verify:

3.1. Load your program into programROM and your constants into dataRAM.

3.2. Run the simulation using single clock steps and record the final value stored in memory.

4. Questions:

Q1: How many total instructions did your program use?

Q2: Trace through your program and write down the value of i and j after each loop iteration.

Q3: Is the final stored value equal to what you would expect mathematically? If not, explain why, considering the register and memory bit width used in this CPU.

Submission:

- Your completed cpu.dig nd others .dig file (including the ROM and RAM contents you configured for this exercise).
- Written answers to Q1–Q3.

Submission at: https://forms.gle/tHAnqGttAysqXxZ86